StudentGator

INTERNSHIP LEARNING PLAN

AI Engineering: From ML to Production

A 4-week customized plan covering Machine Learning, Deep Learning, LLMs, RAG, LangChain/LangGraph, software engineering, FastAPI, Docker, CI/CD and GCP.

Intern	Mohamed
Mentor	Mohammed Orabi
Duration	1 month (4 weeks)
Checkpoints	Weekly 1:1 meeting with Orabi, end of each week
Start date	____ / ____ / 2026

Confidential, internal use

Plan at a glance

Hi Mohamed, welcome to the StudentGator AI team. This plan takes you from classical ML to shipping a production AI application in four weeks. Each week ends with a checkpoint meeting with Orabi to review progress, unblock you, and adjust the plan if needed.

Week	Focus	Deliverable	Checkpoint
1	Machine Learning foundations + Clean Code & SOLID	End-to-end ML notebook on a real dataset	Meeting #1 with Orabi
2	Deep Learning, Transformers, Embeddings, Prompt Engineering + Design Patterns & Async	Embeddings + similarity search notebook, Transformer notes	Meeting #2 with Orabi
3	RAG (naive & advanced), LangChain, LangGraph, LangSmith + FastAPI	RAG API built with FastAPI + LangGraph	Meeting #3 with Orabi
4	Docker, CI/CD, GCP & production deployment	Capstone: RAG app containerized, CI/CD, deployed on Cloud Run	Final review & demo

How to work through this plan

- **Daily rhythm:** about 6–7 focused hours per day, 5 days a week. Mix watching/reading with coding; never watch more than 2 hours without writing code.
- **One GitHub repo:** create *mohamed-ai-learning* with a folder per week. Push notes, notebooks and code daily.
- **Notes:** keep short notes in your own words for every topic. These are reviewed at each checkpoint.
- **Speed:** watch videos at 1.25x–1.5x where comfortable; skip what you already know, but say so at the checkpoint.
- **Stuck for more than 1 hour?** Write down what you tried and ask; don't wait for the weekly meeting.
- **Day 5 of each week** is reserved for the weekly deliverable, review, and the checkpoint meeting.

Important note on the ML book

- Use the **Hands-On Machine Learning** book from this folder: Google Drive link.
- Study **Part I: The Fundamentals of Machine Learning only** (Chapters 1–9).
- **Skip Part II (Neural Networks and Deep Learning)**. Deep Learning is covered in Week 2 with the 3Blue1Brown playlist instead.

WEEK 1 Machine Learning Foundations

Goal: understand the full ML workflow and the classical algorithms, and write clean, maintainable Python.

Day	Topics to cover	Resource
Day 1	• What is ML? Types: supervised, unsupervised, reinforcement • Complete ML pipeline: data collection, cleaning, feature engineering • Train / validation / test split	Hands-On Machine Learning (book, Google Drive) Part I, Ch. 1–2
Day 2	• Handling missing values, categorical features, imbalanced data • Evaluation metrics: accuracy, precision, recall, F1, ROC-AUC • Overfitting vs underfitting, bias vs variance	Hands-On Machine Learning (book, Google Drive) Part I, Ch. 3–4
Day 3	• Linear & logistic regression • Decision trees, random forest • XGBoost / gradient boosting • KNN, SVM	Hands-On Machine Learning (book, Google Drive) Part I, Ch. 4–7
Day 4	• Clustering: K-Means • Feature selection & dimensionality reduction: PCA • Clean Code: readability, naming, separation of concerns • SOLID: SRP, OCP, LSP, ISP, DIP	Hands-On Machine Learning (book, Google Drive) Part I, Ch. 8–9 Clean Code SOLID Principles
Day 5	• Finish the weekly deliverable • Refactor it using Clean Code & SOLID • Checkpoint meeting with Orabi	—

Deliverable

- An end-to-end ML notebook on a public dataset (e.g. Kaggle): EDA, cleaning, feature engineering, at least 3 models compared (e.g. logistic regression, random forest, XGBoost), metrics table and a short conclusion.
- Move the reusable parts (preprocessing, training, evaluation) into clean Python modules that follow SOLID.

Checkpoint 1: meeting with Orabi (end of Week 1)

- Walk through the ML pipeline and explain your model choice and metrics.
- Explain bias vs variance and how you handled over/underfitting.
- Quick code review of your refactored modules (Clean Code & SOLID).
- Bring: your notes, GitHub repo link, and a list of open questions.

WEEK 2 Deep Learning, Transformers & LLMs

Goal: understand how neural networks and Transformers work, how embeddings represent meaning, and when to use prompting vs RAG vs fine-tuning.

Day	Topics to cover	Resource
Day 1	• Neural network basics, perceptron • Activation functions: ReLU, Sigmoid, Tanh, Softmax • Forward propagation, backpropagation, gradient descent • Loss functions; optimizers: SGD, SGD+momentum, AdaGrad, AdaDelta, RMSprop, Adam	3Blue1Brown: Neural Networks playlist (DL, Transformers & LLMs)
Day 2	• CNNs: convolution, pooling; LeNet, AlexNet, ResNet • Transfer learning, data augmentation, image classification pipeline • RNN / LSTM / GRU (concepts)	3Blue1Brown: Neural Networks playlist (DL, Transformers & LLMs)
Day 3	• Attention mechanism, QKV concept, multi-head attention • Transformer architecture, positional encoding, encoder vs decoder • Word embeddings: Word2Vec, GloVe; sentence embeddings	3Blue1Brown: Neural Networks playlist (DL, Transformers & LLMs) Embeddings explained
Day 4	• Prompt engineering techniques • RAG vs prompt engineering vs fine-tuning • Fine-tuning LLM basics: full fine-tuning, LoRA, QLoRA • Design patterns: Singleton, Factory, Builder, Adapter, Facade, Strategy, Observer, DI • Sync vs async: blocking vs non-blocking, concurrency vs parallelism, async/await, event loop, asyncio	Prompt Engineering RAG vs Prompt Engineering vs Fine-tuning Sync vs Async (playlist)
Day 5	• Finish the weekly deliverable • Checkpoint meeting with Orabi	—

Design patterns have no fixed resource. Use refactoring.guru or similar and focus on the AI use cases: Factory for model selection, Strategy for retrievers, Observer for streaming events, Builder for LLM pipelines, Dependency Injection in FastAPI.

Deliverable

- A notebook that embeds a set of sentences/documents, then compares cosine similarity vs dot product for search, with a short write-up of the results.
- One page of notes explaining the Transformer (QKV, multi-head attention, encoder vs decoder) in your own words.
- A small async Python script (asyncio) that calls several APIs in parallel and compares timing vs a sync version.

Checkpoint 2: meeting with Orabi (end of Week 2)

- Explain backpropagation and how attention works (whiteboard style).
- Prompting vs RAG vs fine-tuning: when would you choose each?
- Show the embeddings notebook and the async vs sync comparison.
- Bring: your notes, GitHub repo link, and a list of open questions.

WEEK 3 RAG, LangChain, LangGraph & FastAPI

Goal: build a working retrieval-augmented generation system, improve it with advanced retrieval, orchestrate it with LangGraph, and expose it as an API.

Naive RAG architecture: User Query → Embedding Model → Vector DB → Retrieved Chunks → LLM → Final Answer

Day	Topics to cover	Resource
Day 1	• What is RAG? Embeddings and chunking strategies • Vector DBs: ChromaDB, Qdrant, Pinecone • Similarity search; cosine similarity vs dot product • LangChain: chains, prompt templates, memory, tools, output parsers • Build a simple RAG with LangChain	What is RAG? LangChain
Day 2	• Advanced RAG: query expansion, multi-query retriever, parent-child retriever • Hybrid search: semantic + sparse (BM25), metadata filtering • Re-ranking; cross-encoder vs bi-encoder (accuracy vs speed) • Ensemble retrieval: combining retrievers, weighted strategies	Advanced RAG Techniques (Medium article)
Day 3	• LangGraph: nodes, edges, state management • Linear graphs (A → B → C), conditional edges (A → Router → B/C) • Multi-agent systems, Agentic RAG • LangSmith: observability & tracing	LangGraph LangSmith official documentation
Day 4	• FastAPI: routes, Pydantic models, dependency injection • Async FastAPI APIs, streaming LLM responses, background tasks • Production concepts: streaming, async pipelines, caching, cost optimization	FastAPI full course Sync vs Async (playlist)
Day 5	• Finish the weekly deliverable • Checkpoint meeting with Orabi	—

Deliverable

- A RAG application over a small document set (e.g. StudentGator FAQs or public PDFs): chunking, a vector DB, hybrid search + re-ranking, built as a LangGraph graph with a router node.
- Exposed through FastAPI with an async streaming */chat* endpoint, traced in LangSmith.
- A short comparison: naive RAG vs your advanced RAG on 5–10 test questions.

Checkpoint 3: meeting with Orabi (end of Week 3)

- Live demo of the RAG API and a LangSmith trace.
- Explain your chunking and retrieval choices and the naive vs advanced comparison.
- Code review: project structure, design patterns used, async correctness.
- Bring: your notes, GitHub repo link, and a list of open questions.

WEEK 4 DevOps, Cloud & Production Deployment

Goal: containerize the Week 3 application, automate testing and deployment, and run it on Google Cloud.

Day	Topics to cover	Resource
Day 1	• What is Docker? Containers vs VMs, Docker architecture • Images & containers, writing Dockerfiles • Environment variables, volumes & networking, Docker Compose	Docker full course (watch from start to 8:36:00) Watch start → 8:36:00 (split over Days 1–2)
Day 2	• Finish the Docker course • Containerize FastAPI apps, ML models and the RAG application • Multi-stage Docker builds	Docker full course (watch from start to 8:36:00)
Day 3	• What is CI/CD? Continuous integration vs continuous deployment • GitHub Actions: automated testing, linting, build & deployment pipelines • Secrets management; rolling vs blue/green deployment; monitoring pipelines	CI/CD with GitHub Actions
Day 4	• GCP fundamentals: Compute Engine, IAM & permissions, networking basics • Storage: Cloud Storage, Persistent Disk; billing & cost management • VM setup & SSH, Docker on GCP, Artifact Registry, Cloud Run • Deploying ML APIs, RAG systems and AI chatbots	Google Cloud Skills Boost learning path
Day 5	• Finish the capstone, write the README • Final review & demo with Orabi	—

Capstone deliverable

- The Week 3 RAG app with a Dockerfile (multi-stage) and docker-compose for local runs.
- A GitHub Actions pipeline that runs linting and tests, builds the image and pushes it to Artifact Registry.
- The app deployed on Cloud Run with secrets handled properly, plus a README explaining the architecture, how to run it, and the estimated cost.

Checkpoint 4: meeting with Orabi (end of Week 4)

- Final demo: the deployed app on Cloud Run and the CI/CD pipeline run.
- Explain production concerns: background workers, async processing, model serving, rate limiting, caching.
- Retrospective: what went well, what to deepen next, and goals after the internship.
- Bring: your notes, GitHub repo link, and a list of open questions.

Resources

All resources in one place. Links are clickable.

Area	Resource	Link
Machine Learning	Hands-On Machine Learning (book, Google Drive)	drive.google.com/drive/folders/12p0IpToLFE-3G0aCZM0S...
Deep Learning, Transformers & LLMs	3Blue1Brown: Neural Networks playlist (DL, Transformers & LLMs)	www.youtube.com/watch?v=aircAruvnKk&list;=PLZHQObOWTQ...
	Embeddings explained	youtu.be/hVM8qGRTaOA?si=9Id20sW-TbUUaxh7
	Prompt Engineering	www.youtube.com/watch?v=2BpCk4d2Cc0
	RAG vs Prompt Engineering vs Fine-tuning	youtu.be/ahnGLM-RC1Y?si=fOoKoVh9WjPQEtdH
RAG	What is RAG?	www.youtube.com/watch?v=T-D1OfcDW1M&t;=18s
	Advanced RAG Techniques (Medium article)	medium.com/data-science-collective/advanced-rag-tech...
LangChain, LangGraph & LangSmith	LangChain	youtu.be/qAF1NjEVHhY?si=Ox6lYMI7hpervcYt
	LangGraph	youtu.be/1w5cCXIh7JQ?si=LXpO1m-eb8HVRsXp
	LangSmith official documentation	docs.smith.langchain.com/
Software Engineering	Clean Code	youtu.be/vhdUyGs_f6c?si=E5LDtT1-EKt_E_md
	SOLID Principles	www.youtube.com/watch?v=pTB30aXS77U
	Sync vs Async (playlist)	www.youtube.com/watch?v=cZ3iHLIDzVI&list;=PLhTjy8cBIS...
	FastAPI full course	youtu.be/tiBeLLv5GJo?si=en2YiPgcgw2gmjOX
DevOps & Cloud	Docker full course (watch from start to 8:36:00)	youtu.be/PrusdhS2lmo?si=01cKK4MPP4UiCpSb
	CI/CD with GitHub Actions	youtu.be/7gJFHjXscr8?si=9oODNvm9ZugCtWCe
	Google Cloud Skills Boost learning path	www.skills.google/paths/4681/course_templates/60

Checkpoint log

To be filled in during each meeting with Orabi.

#	Date	Progress / status	Feedback & next actions
1			
2			
3			
4			